# CCE v2 · Admin Module

Product Requirements Document & Hands-on Onboarding Guide

For new engineers joining the CCE team · 29 October 2026

## Contents

1. How to use this document . . . 2
2. Product overview . . . 2
3. Users and key scenarios . . . 3
4. Key concepts . . . 4
5. System context . . . 5
6. Technology stack . . . 5
7. Backend requirements (cce-admin-service) . . . 6
8. Frontend requirements (cce-admin-ui) . . . 8
9. Non-functional requirements . . . 10
10. Hands-on development plan . . . 10
11. Definition of done . . . 12
12. Risks and open questions . . . 13
Appendix A. Endpoint summary . . . 13
Appendix B. Error codes . . . 14

# 1. How to use this document

This PRD turns the design of the admin module for **CCE v2** into a buildable plan for new team members. The module's repositories are at the design stage with no code yet, so you will be building it from an empty scaffold. That is deliberate: by the end you will have touched every layer of the platform's access-control story, from Keycloak tokens to the gateway's permission check to a role-aware sidebar.

Read sections 2 to 6 for the **what and why**. Sections 7 and 8 are the **requirements** you will be checked against. Section 10 is the **hands-on plan**: a sequence of milestones, each small enough to finish and demo in a day or two.

> **Golden rule.** The team's design documentation for the admin module is the source of truth for contracts, schema and behaviour. This PRD summarises and sequences it; it does not override it. If you find a conflict, raise it with the team rather than picking one silently.

## Components involved

| Component | Role in CCE v2 |
|---|---|
| cce-admin-service | New backend: users, roles, menus, API permissions, Keycloak provisioning, audit |
| cce-admin-ui | New admin console for platform administrators |
| cce-insights-ui | Existing analytics console whose sidebar this module controls; also the template for the UI stack |
| gateway-service | Existing JWT validation and `api_permissions` enforcement; not being redesigned |
| Keycloak | Identity provider: issues tokens, holds user accounts and realm roles |

# 2. Product overview

## Problem

`cce-insights-ui` renders the same static sidebar to every authenticated user and does not read roles from the token. The gateway can already block API calls per role, but nothing manages who has which role, which screens a role should see, or which APIs a role may call. Today that would mean hand-editing Keycloak and a database table that the gateway reads directly.

## Solution

A two-part admin module:

- **cce-admin-service**, a Spring Boot backend that owns users, roles, menus and API permissions, keeps the database and Keycloak consistent, and exposes `GET /me/menus` so consoles can render a role-aware sidebar.
- **cce-admin-ui**, a React admin console where a platform administrator creates users, defines roles, picks the menus and API permissions for each role, and assigns roles to users.

## Goals for CCE v2

- A user logging in to `cce-insights-ui` sees only the menus their role permits.
- The gateway lets a user call only the APIs their role permits, configured through the admin console rather than SQL.

- Every mutating admin action is recorded in `audit_log` so “who gave this user access, and when” is answerable.
- Users, roles and role assignments stay consistent between the local database and Keycloak, and partial failures are visible rather than silent.

### Out of scope for CCE v2

| Not in this release | Why / where it goes later |
|---|---|
| Audit log viewer screen | Backend writes `audit_log` but has no read endpoint yet. Needs a backend addition first. |
| Global API-permission catalog | Permissions are only viewed and edited per role, matching the API shape (no cross-role `GET /permissions`). |
| Facility-scoped data enforcement | `users.facility_scope` is stored and displayed only. Enforcing it is a future extension. |
| Deriving permissions from menus | Menus and permissions are independent axes in this release. Auto-suggesting permissions is a future enhancement. |
| Hard delete of users | Deactivation (`INACTIVE`) is terminal, for audit continuity. |
| Optimistic UI | Writes fan out to Keycloak and the gateway and can fail midway, so the UI always waits. |
| Role-driven nav in the admin console itself | The admin UI sidebar is static; authorization is enforced by API permissions (403 shown inline). |

## 3. Users and key scenarios

| Persona | Who | What they need |
|---|---|---|
| Platform administrator | Operates the CCE platform (e.g. SUPER_ADMIN) | Onboard and deactivate users, define roles, map menus and API permissions, recover from sync failures |
| Console user | Clinician, facility admin, national viewer using cce-insights-ui | See only the screens relevant to their role; be blocked from APIs they should not call |
| Integrating engineer | Other CCE teams | A stable, documented API (OpenAPI/Swagger) and predictable error codes |

### Core scenarios

| # | Scenario | Outcome |
|---|---|---|
| S1 | Console user logs in to cce-insights-ui | Sidebar is built from `GET /admin/v1/me/menus`, not a static list |
| S2 | Admin onboards a new user with one or more roles | Keycloak account, local row, realm-role assignments and audit entry, all in one submit |
| S3 | Admin creates and configures a role | Role created, menus selected, API permissions saved, gateway sync triggered so the realm role exists |
| S4 | Admin changes a user's roles | Full replacement set sent; backend diffs and assigns/removes Keycloak realm roles |
| S5 | Admin deactivates a user | Keycloak account disabled, status INACTIVE, role assignments kept for re-activation |
| S6 | Gateway sync fails after a permission save | UI shows “sync failed, retry”; admin uses Re-sync before assigning the role |

# 4. Key concepts

Get these straight before writing code. Most design decisions follow from them.

| Term | Meaning |
|---|---|
| User | A person who can sign in to a CCE console. Mirrored 1:1 with a Keycloak user; joined to the token by `keycloak_id` = JWT `sub`. No credentials stored locally. |
| Role | A named grouping of access, e.g. `FACILITY_ADMIN`. `UPPER_SNAKE_CASE`. Immutable name after creation. |
| Menu | One screen of cce-insights-ui. `NAV` items appear in the sidebar; `DETAIL` items are drill-down routes that inherit their parent's visibility. |
| Permission | One `(HTTP method, URI pattern)` rule a role may call, stored in `api_permissions` and enforced by the gateway. |
| Role → Menu | Soft control: what the UI renders. Wrong config is cosmetic. |
| Role → Permission | Hard control: what the gateway proxies. Wrong config is a security issue. |

## One identifier, three systems

A role's `roles.name` is used unchanged as the Keycloak realm role name and as `api_permissions.permission_name`. That is why the name cannot change after creation and why the create-role form must warn before submit.

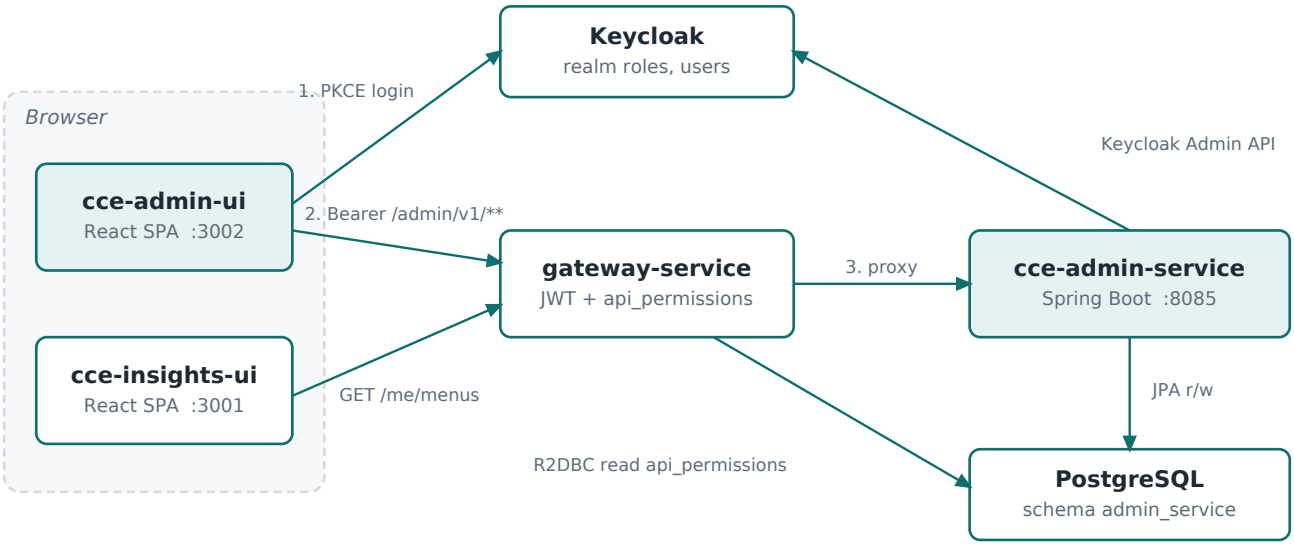
## Two independent axes

Granting a role the Compliance menu does **not** grant it the compliance APIs, and vice versa. Admins configure both on the same Role Detail screen so neither is forgotten.

## Who provisions what in Keycloak

| Responsibility | Owner |
|---|---|
| Create/delete realm role definitions | gateway-service, via its existing `POST /api/permissions/sync-to-keycloak`, sourced from `api_permissions` |
| Create the Keycloak user account | cce-admin-service (Keycloak Admin REST API) |
| Assign/remove realm roles on a user | cce-admin-service |
| Password reset / required actions | cce-admin-service |

# 5. System context




Both consoles are peer Keycloak-authenticated SPAs behind the same `gateway-service`. Neither talks to the database or to Keycloak's admin API directly. The gateway validates the JWT, checks the caller's role against `api_permissions`, and proxies `/admin/**` to the admin service, which **re-validates the JWT** itself (defense in depth) and also rejects tokens whose local user is missing or not `ACTIVE`.

> **Fixed constraints you cannot change.** The gateway already reads a Postgres database named `admin_service` and a table named `api_permissions` with a specific column shape. Check gateway-service's own migration doc before altering that table; a mismatch breaks a service this team does not own.

# 6. Technology stack

## cce-admin-service

| Concern | Choice |
|---|---|
| Language / framework | Java 21, Spring Boot 3.x, Spring Web MVC (blocking; this is low-throughput CRUD) |
| Persistence | PostgreSQL schema `admin_service`, Spring Data JPA + Hibernate, Flyway migrations in `src/main/resources/db/migration` |
| Security | `spring-boot-starter-oauth2-resource-server` with JWKS; roles from configurable `JWT_ROLES_CLAIM_PATH` |
| Keycloak | `org.keycloak:keycloak-admin-client` |
| API docs | springdoc-openapi (OpenAPI 3 + Swagger UI) |
| Build / run | Maven, multi-stage Docker image, port `8085`, Jenkins CI |

Package layout: `controller/ service/ repository/ entity/ security/ client/ audit/ config/`. Controllers only shape HTTP; side effects to Keycloak and the gateway live behind `client/`; audit writes live behind `audit/`.

### cce-admin-ui

| Concern | Choice |
|---|---|
| Core | React 18, TypeScript 5, Vite 6, React Router 7, npm 10+ |
| Server state | TanStack Query 5 (no optimistic updates) |
| Auth | `keycloak-js` 26, Authorization Code + PKCE (S256), client id `cce-admin-ui`, `updateToken(70)` every 60 s, no auth-bypass flag |
| HTTP | Typed wrapper over native `fetch` in `src/api/client.ts`; one module per resource |
| UI | Tailwind CSS 4, Heroicons 2, date-fns 4, TanStack Table 8 |
| Forms | React Hook Form 7 + Zod 3 |
| Testing | Vitest + Testing Library + MSW |
| Build / run | Docker: `node:20-alpine` build → `caddy:2-alpine` runtime, port `3002` |

Folder layout: `pages/ components/ hooks/ api/ auth/ schemas/ context/`. Pages compose hooks and components and never call `fetch` directly.

# 7. Backend requirements (cce-admin-service)

## 7.1 Foundations

| ID | Requirement | Acceptance criteria |
|---|---|---|
| B-01 | Flyway migrations create all seven tables (`users, roles, menus, role_menus, user_roles, api_permissions, audit_log`) with UUID PKs and timestamps per the data dictionary. | Fresh database migrates cleanly; indexes listed in the data dictionary exist; `pgcrypto` enabled. |
| B-02 | Seed the 12 cce-insights-ui menus and a system `SUPER_ADMIN` role. | Seed matches the data-dictionary table exactly, including `Patients` as its own top-level NAV and the two DETAIL entries. |
| B-03 | Uniform response envelope and error body. | Success: `{data, pagination?}`. Error: `{error:{code,message}}`. All codes from the API reference are produced where applicable. |
| B-04 | List endpoints paginate with `page` and `pageSize`. | Default 20, max 100; `pagination` block returned with total. |
| B-05 | `GET /health` is unauthenticated. | Returns `{status:"UP"}` with best-effort DB status. |
| B-06 | OpenAPI spec and Swagger UI generated from controllers. | Spec covers every endpoint in the API reference. |

## 7.2 Security

| ID | Requirement | Acceptance criteria |
|---|---|---|
| B-10 | Validate every request's JWT independently of the gateway. | Signature (JWKS), `iss`, `aud`, expiry checked; invalid → 401 `UNAUTHENTICATED`. |
| B-11 | Read roles from configurable claim path. | `JWT_ROLES_CLAIM_PATH` default `realm_access.roles`, dot-notation. |
| B-12 | Resolve `sub` to a local user and check standing. | Valid token but no local user, or status ≠ ACTIVE → 403 `FORBIDDEN`. |
| B-13 | No secrets in the repo. | All credentials come from env vars; local dev defaults never used in shared environments. |

## 7.3 Current user

| ID | Requirement | Acceptance criteria |
|---|---|---|
| B-20 | `GET /me` returns the caller's profile and role names. | Shape: id, username, roles[], facilityScope. |
| B-21 | `GET /me/menus` returns the menu tree for the caller's roles. | Union across all caller roles; only `is_active` rows; NAV and DETAIL both included; children nested under parent; ordered by `sort_order`. |

## 7.4 Users

| ID | Requirement | Acceptance criteria |
|---|---|---|
| B-30 | `GET /users` with filters `status, roleId, facilityScope, q`. | Search matches name or email; paginated. |
| B-31 | `POST /users` creates the Keycloak account, assigns realm roles, inserts the user and `user_roles`, writes audit. | Order follows flow diagram 2. Returns 201. Duplicate email → 409 `EMAIL_IN_USE`. |
| B-32 | Partial failure in user creation is surfaced, not retried. | Returns 502 `KEYCLOAK_SYNC_FAILED`; `keycloak_id` recorded in `audit_log` for manual reconciliation. |
| B-33 | `GET /users/:id`, `PUT /users/:id` (firstName, lastName, email, facilityScope). | Unknown id → 404 `USER_NOT_FOUND`. |
| B-34 | `PATCH /users/:id/status` also enables/disables the Keycloak account. | INACTIVE keeps `user_roles` rows; re-activating restores access. Audit `USER_DEACTIVATED`. |
| B-35 | `PUT /users/:id/roles` replaces the full role set. | Backend diffs old vs new and assigns/removes only the changed realm roles; audit `USER_ROLES_UPDATED`. |
| B-36 | `POST /users/:id/reset-password` triggers a Keycloak reset action. | No password ever passes through the service. |

### 7.5 Roles, menus and permissions

| ID | Requirement | Acceptance criteria |
|---|---|---|
| B-40 | Role CRUD. `name` immutable; only displayName/description editable. | Duplicate → 409 `DUPLICATE_ROLE_NAME`. Delete blocked: 409 `ROLE_IN_USE` or `SYSTEM_ROLE_IMMUTABLE` (checked in service layer, not via FK error). |
| B-41 | `GET/PUT /roles/:id/menus` (replace semantics). | Granting a DETAIL without its parent NAV is allowed but returns a validation warning. Audit `ROLE_MENUS_UPDATED`. |
| B-42 | `GET/PUT /roles/:id/permissions` (replace semantics). | Rows stored with `permission_name = roles.name`; gateway sync called automatically afterwards. Audit `ROLE_PERMISSIONS_UPDATED`. |
| B-43 | `POST /roles/:id/sync` calls gateway `POST /api/permissions/sync-to-keycloak`. | Sync failure → 502 `KEYCLOAK_SYNC_FAILED`; DB changes remain saved. |
| B-44 | Menu CRUD (`GET/POST/PUT/DELETE /menus`). | GET returns a tree. Delete also removes `role_menus` rows. Duplicate key → 409 `DUPLICATE_MENU_KEY`. |

### 7.6 Audit

| ID | Requirement | Acceptance criteria |
|---|---|---|
| B-50 | Every mutating endpoint writes an append-only `audit_log` row. | Actor, action, entity type/id, before/after JSON, IP. A test asserts one row per mutation. |

## 8. Frontend requirements (cce-admin-ui)

### 8.1 Shell, auth and API client

| ID | Requirement | Acceptance criteria |
|---|---|---|
| F-01 | `initAuth()` runs before render; login is required. | PKCE S256 against client `cce-admin-ui`; token refreshed via `updateToken(70)` on a 60 s interval. |
| F-02 | Typed API client in `src/api/client.ts`. | Base `${VITE_API_BASE_URL}/admin/v1`; sends Bearer and `Accept: application/json`; unwraps envelope; throws `ApiError` with `status` and `code`. |
| F-03 | App shell: static sidebar (Users, Roles, Menus) and header from `GET /me`. | Header shows signed-in username and roles; sidebar does not depend on `/me/menus`. |
| F-04 | 403 handling. | `FORBIDDEN` shown inline on the screen that made the call, not as a global error page. |
| F-05 | Distinct sync-failure state. | `KEYCLOAK_SYNC_FAILED` has its own message explaining partial state and next step, never a generic toast. |
| F-06 | Mutations wait for the server. | No optimistic updates; buttons show pending state; queries invalidated on success. |

## 8.2 Screens

| ID | Requirement | Acceptance criteria |
|---|---|---|
| F-10 | **Users list** `/users`: username, email, status, roles, facility scope, last login. | Filters (status, role, facility, search) map to query params; server-side pagination via TanStack Table; row actions view/edit, activate/deactivate, reset password. |
| F-11 | **User create/edit** `/users/new`, `/users/:id`. | RHF + Zod validation. Create posts once (no separate sync step). Edit uses PUT profile, PUT roles (full set), PATCH status, POST reset-password as separate actions. |
| F-12 | **Roles list** `/roles` with inline create. | System roles show a lock and cannot be deleted/renamed. Create form warns that `name` is permanent. Client validates UPPER_SNAKE_CASE. |
| F-13 | **Role detail · Menus tab** `/roles/:id`. | Checkbox tree from `GET /menus` with current selection from `GET /roles/:id/menus`. DETAIL-without-parent shows a warning. Save sends full set and is disabled when unchanged. |
| F-14 | **Role detail · Permissions tab**. | Editable rows (method, URI pattern, description, category); Save sends full set; shows “synced” or “sync failed, retry”; manual Re-sync button. |
| F-15 | **Menus catalog** `/menus`. | Tree view of label, path, icon, type, active; add, edit, deactivate entries. |

## 8.3 Environment and scripts

| Variable | Default | Notes |
|---|---|---|
| `VITE_KEYCLOAK_URL` | `${origin}/auth` | Keycloak base URL |
| `VITE_KEYCLOAK_REALM` | required | Same realm as the service and cce-insights-ui |
| `VITE_KEYCLOAK_CLIENT_ID` | `cce-admin-ui` | Own client, separate from cce-insights-ui |
| `VITE_API_BASE_URL` | `""` | Prefixed to `/admin/v1/*` |

Scripts: `npm run dev` (port 3002), `build` (`tsc -b` then Vite), `test`, `test:ci`, `lint`. `VITE_*` values are inlined at build time, so each environment needs its own image.

# 9. Non-functional requirements

| Area | Requirement |
|---|---|
| Consistency | Keycloak calls can partially fail. Operations are ordered as in the flow diagrams; partial state is surfaced as 502 and recorded for reconciliation rather than blindly retried. |
| Ordering | A role's realm role must exist in Keycloak (gateway sync) before it is assigned to any user. |
| Auditability | Every mutation is reconstructable from `audit_log`. This is a compliance platform; treat this as a correctness requirement. |
| Security | Defense in depth (service re-validates JWT); least-privilege Keycloak service account; no secrets committed; admin UI never bypasses auth. |
| Network | In every non-local environment both apps reach the service only through gateway-service; the service is not internet-facing. |
| Config parity | `JWT_ISSUER`, `JWT_AUDIENCE`, `JWT_ROLES_CLAIM_PATH` match gateway-service for the same realm. |
| Testability | Backend: service-layer tests with Keycloak and gateway clients mocked. Frontend: MSW-mocked API, forms tested through user input and submit. |

# 10. Hands-on development plan

The module splits naturally into a backend track and a frontend track. Because the API is designed contract-first, the frontend track can start immediately against MSW mocks and switch to the real service once backend milestones land. Pairs work well: one person per track, swapping for review.

> **Suggested local setup** (not prescribed by the design docs; agree it with your lead): a `docker-compose.yml` with Postgres 16 and Keycloak, a local realm containing the `cce-admin-ui` public client (redirect `http://localhost:3002/*`) and a confidential service-account client for the backend. Locally the UI can proxy `/admin/v1/*` straight to `localhost:8085` via the Vite dev proxy. Stub the gateway sync call behind an interface so it can be a no-op or a mock server locally.

## Milestones

| # | Milestone | Covers | Demo / exit check |
|---|---|---|---|
| M0 | Local environment | Postgres, Keycloak realm, clients, a test admin user | Log in to Keycloak account console as the test admin |
| B1 | Service skeleton | B-01…B-06, B-10…B-13 | Migrations + seed run; /health UP; protected endpoint rejects a bad token with 401 |
| B2 | Roles and menus | B-40, B-41, B-44, B-50 | Create a role, map menus via Swagger; audit rows appear; DETAIL-only warning returned |
| B3 | Users + Keycloak | B-30…B-36 | POST /users creates a Keycloak user with realm role; deactivate disables it in Keycloak |
| B4 | Permissions + sync | B-42, B-43 | Permissions saved; sync client called; simulated failure returns 502 |
| B5 | Current user | B-20, B-21 | Two roles with overlapping menus produce the correct merged tree |
| F1 | UI skeleton | F-01…F-06 | Login redirect works; header shows /me; ApiError surfaces codes in a test |
| F2 | Roles screens | F-12, F-13 | Create role with name warning; menus tree saves full set; warning on DETAIL-only |
| F3 | Permissions tab | F-14 | Save shows synced; MSW 502 shows retry state; Re-sync works |
| F4 | Users screens | F-10, F-11 | Paginated filtered list; create and edit flows; 502 state on create |
| F5 | Menus catalog | F-15 | Add and deactivate a menu entry |
| E2E | Integration | S1–S6 | Run all six scenarios against the real service locally |

## Milestone notes

### B1 · Service skeleton

- Generate a Spring Boot 3 / Java 21 Maven project; add Web, Data JPA, Flyway, Postgres, OAuth2 Resource Server, Validation, springdoc.
- Write `V1__schema.sql` straight from the data dictionary and `V2__seed.sql` for menus and SUPER_ADMIN. Have a reviewer diff your columns against the doc.
- Add a `@RestControllerAdvice` that maps domain exceptions to the error codes table. Do this early; every later milestone depends on it.
- Add a filter or authentication converter that resolves `sub` to a local user and enforces ACTIVE (B-12).

### B3 · Users + Keycloak

- Wrap `keycloak-admin-client` behind a `KeycloakGateway` interface in `client/` so service tests can mock it.
- Follow flow diagram 2 order exactly. Write a test where the DB insert fails after Keycloak succeeds and assert the 502 and the audit row.
- For `PUT /users/:id/roles`, compute added and removed sets and touch only those realm roles.

### B4 · Permissions + sync

- Replace rows in one transaction, then call the gateway sync. If sync fails, keep the DB change and return 502 (flow diagram 4).
- Confirm the `api_permissions` column types with gateway-service's migration doc before writing the migration.

### F1 · UI skeleton

- Scaffold with Vite's React + TS template; add Tailwind 4, React Router 7, TanStack Query/Table, RHF, Zod, keycloak-js, MSW.
- Build `auth/` first (init, `getToken()`, context), then `api/client.ts` with `ApiError`, then MSW handlers mirroring the API reference examples.
- Every hook lives in `hooks/` and wraps one `api/` function. Pages never import `api/` directly.

### F2–F4 · Forms and replace semantics

- Diff against the last-fetched state **only** to enable or disable Save. The payload is always the complete set.
- Mirror backend validation in Zod (UPPER_SNAKE_CASE role names, email format, required fields) but still render server errors by `error.code`.
- Give `KEYCLOAK_SYNC_FAILED` its own component; it is reused on the user-create form and the permissions tab.

## 11. Definition of done

A milestone is done when all of the following hold:

- Every requirement ID listed for the milestone meets its acceptance criteria, and the PR description lists those IDs.
- Tests cover the happy path and at least one error code per endpoint or screen touched. Backend: service layer with mocked Keycloak and gateway clients. Frontend: Testing Library + MSW.
- Mutating endpoints are shown to write an `audit_log` row.
- No secrets, `.env` files or local Keycloak exports committed.
- `npm run build` / `npm run lint` / `mvn verify` pass in CI.
- Any deviation from the design docs is either fixed or raised as a doc change in the same PR.

# 12. Risks and open questions

| Item | Notes |
|---|---|
| Gateway column types | `api_permissions` shape is owned by gateway-service's consumer. Confirm against its migration doc before writing B1. |
| Bootstrap admin | The first SUPER_ADMIN must exist in Keycloak and locally, with `api_permissions` rows for `/admin/v1/**`. Decide how this is seeded per environment. |
| Bootstrap of /me/menus | Every console user's role needs a permission row for `GET /admin/v1/me/menus`, or cce-insights-ui cannot load its sidebar. |
| Manual reconciliation | 502 partial states rely on an operator reading `audit_log`. There is no UI for this in CCE v2. |
| Discoverability of 403s | Admin UI shows nav items the caller may not be able to use. A `GET /me`-driven show/hide is a noted follow-up. |
| last_login_at | Doc says it is updated by a lightweight callback. The mechanism is not yet specified. |
| cce-insights-ui integration | Rendering the sidebar from `/me/menus` is a separate frontend change in that repo. |

# Appendix A. Endpoint summary

Base path `/admin/v1`. All require a Bearer token except health.

| Method | Path | Purpose |
|---|---|---|
| GET | /me | Caller profile and role names |
| GET | /me/menus | Role-aware menu tree for consoles |
| GET / POST | /users | List (filtered, paginated) / create with Keycloak provisioning |
| GET / PUT | /users/:id | Fetch / update profile |
| PATCH | /users/:id/status | ACTIVE, INACTIVE, LOCKED (mirrors to Keycloak) |
| PUT | /users/:id/roles | Replace role set |
| POST | /users/:id/reset-password | Trigger Keycloak reset |
| GET / POST | /roles | List / create |
| GET / PUT / DELETE | /roles/:id | Fetch / update display fields / delete (guarded) |
| GET / PUT | /roles/:id/menus | Read / replace menu set |
| GET / PUT | /roles/:id/permissions | Read / replace permissions (auto-sync) |
| POST | /roles/:id/sync | Manual gateway sync |
| GET / POST | /menus | Catalog tree / create |
| PUT / DELETE | /menus/:id | Update / delete (removes role_menus) |
| GET | /health | Unauthenticated health check |

# Appendix B. Error codes

| HTTP | Code | When |
|---|---|---|
| 400 | VALIDATION_ERROR | Body or query failed validation |
| 401 | UNAUTHENTICATED | Missing, invalid or expired token |
| 403 | FORBIDDEN | Role lacks permission, or local account not ACTIVE |
| 404 | USER_ / ROLE_ / MENU_NOT_FOUND | Unknown id |
| 409 | DUPLICATE_ROLE_NAME / DUPLICATE_MENU_KEY / EMAIL_IN_USE | Unique conflict |
| 409 | ROLE_IN_USE | Delete of a role still assigned to users |
| 409 | SYSTEM_ROLE_IMMUTABLE | Modify or delete of an is_system role |
| 502 | KEYCLOAK_SYNC_FAILED | Keycloak or gateway sync failed; state partially applied |